

BTRON Concurrency Tiers. MP/AMP/SMP/Async, Media RTP and the InterCore in the Calculus of Inductive Constructions, with Modal Extensions for Non-Determined Multicore Parallelism

Maxim Sokhatsky (Namdak Tonpa)

Synrc Research Center / Groupoid Infinity / BTRON Cleanroom Working Group

Kyiv, Ukraine

maxim@synrc.com

September 2026

Abstract

The cleanroom reconstruction of B-System (BTRON 3.20) has produced, alongside the C kernel, a second artefact that is easy to overlook: a body of mechanical mathematics in which the concurrency, media, and inter-core subsystems are not described but *defined*, and in which their properties are not argued for but *computed*. This paper presents that body of work as a single mathematical model. We take as our object language the Calculus of Inductive Constructions, and as our single rule of inference the judgemental equality $t = u$ discharged by **reflexivity**. Four claims are made. First, that the apparently heterogeneous tiers of the BTRON scheduling model—multi-processing, asymmetric multiprocessing, symmetric multiprocessing, and the asynchronous I/O plane—are four instances of one algebraic signature, a total state transformer $S \rightarrow S \times R$ over a table-shaped bus, differing only in which coordinates of the state their guards read. Second, that the media stack, from NuStreams abstraction layers through RTP packetisation and loss accounting to the symmetric multi-plane pipeline, is equationally a *fold over a watermark*: every streaming invariant we state, including the sequence-number, jitter, and residence-time laws, reduces to arithmetic on counters that are themselves derived as lengths and differences of lists, never stored. Third, that the InterCore protocol—**pub**, **sub**, **spawn**, **snd**, **rcv** over a star of single-producer queues—admits a broker-free construction in which the multi-consumer view is a pure function of the producer views, so that the entire sharing discipline collapses to one equation about cursor ownership and its corollary, the absence of mutual exclusion and therefore of priority inversion between equal-priority planes. Fourth, that genuine, non-determined multicore parallelism—not an interleaving chosen by a simulator but the modal fact that *any*

of several next steps may occur—is representable inside the same kernel as an internal box-and-diamond pair over the step alphabet, with trace equivalence induced by a commutation lemma between independent calls, and with platform modes as indices of a small Kripke frame in which an unbuildable mode yields *SKIP*, never a vacuous *PASS*. Every number quoted is produced by an executable harness: the assembled InterCore theory discharges 228 closed obligations with **Print Assumptions** reporting no axioms, mirrored by 75 oracle checks over all $8^6 = 262,144$ runs of six call sites, each site drawn independently from the eight operations admitted by the API.

Keywords—BTRON, Calculus of Inductive Constructions, equational reasoning, formal semantics, lock-free queues, InterCore, RTP, NuStreams, SMP, modal logic, model checking, theorem proving.

1 Introduction: The Model Is the Document

Operating-system specifications are written in a language designed for the purpose of being read, and are then implemented in a language designed for the purpose of being executed. The gap between the two is where real-time defects live. The BTRON cleanroom programme [2] has, from its first commits, maintained a third artefact: a mechanically checked definition of the subsystem under construction, written in the Rocq/Coq proof assistant [3,4], together with an executable oracle of the same definition in OCaml [5], run on every change by a single shell harness.

This paper argues a narrow methodological thesis with wide consequences: *for the classes of properties a real-time kernel actually needs, the whole of verification reduces to equational reasoning*, and the interesting engineering deci-

sions are the ones that keep the equations in the reducing fragment. Not induction over traces, not bisimulation, not a separation logic of permissions—a calculus of rewriting totals over finite tables, closed by decidability of arithmetic. The scope is deliberately narrower than a whole-kernel verification [9, 13]: it is the subsystem boundary where BTRON’s real-time claims are actually made.

Four model families are treated:

1. the four *concurrency tiers* of the BTRON scheduling model—MP, AMP, SMP, Async (§3);
2. the *media* stack, from the NuStreams abstraction layer to RTP reception and the symmetric multi-plane pipeline (§4);
3. the **InterCore** protocol, the five-call message API between cores of the target four-core machine (§5);
4. the *modal extension* that lets one flat functional definition speak about genuinely non-determined parallel execution (§6).

Section 2 fixes the kernel and the five modelling idioms that recur in all four families. Section 7 describes the harness that keeps the mathematics and the machine honest, and states the one policy that makes the harness meaningful: *SKIP is not PASS*.

1.1 Why Equations, and Not Something Heavier

The properties demanded of a soft- and hard-real-time kernel are, with few exceptions, of four grammatical forms:

$$\begin{aligned}
 (\text{conservation}) \quad & n_{\text{in}} = n_{\text{out}} + n_{\text{held}} + n_{\text{lost}} \\
 (\text{monotonicity}) \quad & x(s') \leq x(s) \quad \text{or} \quad x(s) \leq x(s') \\
 (\text{invariance}) \quad & \pi_j(s') = \pi_j(s) \quad (j \neq i) \\
 (\text{provenance}) \quad & v = \pi_k(s') \Rightarrow v \in \pi_{\text{hist}}(s).
 \end{aligned} \tag{1}$$

Each of these is an *equation* between naturals or booleans computed from one state, or between two states related by one call. None of them needs a logic of resources. The consequence for the modelling language is severe and beneficial: the specification language can be the programming language, so long as the programming language is total.

1.2 Contributions

1. A single signature—total state transformer over a table-shaped bus, with derived accounting—is shown to cover all four concurrency tiers, the media stack, and InterCore (Table 1).
2. The watermark operator `min_fold` is identified as the shared algebraic content of multi-consumer retention in both media (§4) and InterCore (§5.3): residence, freeing, and the “no over-fill” theorem are the same three lemmas in two guises.

3. Cursor ownership is proved, not assumed, to be the mutex-elimination theorem of the InterCore star (§5.2), and the resulting absence of priority inversion between equal-priority INPUT and UI planes is stated as a corollary about the guards of the five calls.
4. Non-determined parallelism is internalised (§6) as a box/diamond pair over the step alphabet, computed by `forallb/existsb` on the same tables the model uses, with trace equivalence induced by commutation of independent calls—a machine-checked partial-order reduction.
5. An engineering account of what breaks when totality is taken seriously: the `Nat.sub` unfolding trap, the shadowing of `snd`, unary reduction of shipped constants (§2.6, §7).

2 The Equational Kernel

2.1 Judgemental Equality as the Only Discharge Rule

The CIC identifies propositions with types and proofs with programs; a *theorem* about a data structure is an inhabitant of a dependent product, and the overwhelming majority of inhabitants we construct are the single term `reflexivity`:

Listing 1: A frame lemma: writing one writer leaves the other writers untouched.

```

Lemma at_w_other_w : forall b i (w:writer) j, i <> j ->
  b_w (bus_w b i w) j = b_w b j.
Proof. reflexivity. Qed.

```

The point of `reflexivity` is that it checks *convertibility*—equality as computed by the kernel’s reduction machine—not equality as argued. When a lemma closes with `reflexivity`, the fact was already true by definition, and the specification has been arranged so that the fact is worth stating. The craft of the model consists in arranging for as much as possible to be definitional. Where computation is too shallow, we descend one step: `lia` for linear arithmetic over the successors and truncated subtractions that conversion leaves behind, and induction only where a fold genuinely recurses.

2.2 Totality, or the Absence of Admitted

Every function in the model is *total and terminating*, including the ones that the C implementation would signal as errors. The C `nth` is a partial function; the model’s is `nth k 1 d`, which returns a default. The C ring buffer write on a full ring is undefined behaviour; the model’s returns a receipt. The discipline has three consequences we exploit throughout:

- there are no preconditions, hence no hypothesis book-keeping, hence no *vacuous-truth* bugs in the specification;

- every function is executable, so the identical text serves as the OCaml oracle (§7);
- the model cannot express “the kernel crashed here”, which is precisely the abstraction level wanted for a specification of a kernel that must not crash.

2.3 Tables: Finite Maps as Functions

The bus is a record of three *total* functions from dense small integers to cells, plus the one non-indexed coordinate:

```
Record bus : Type := mk_bus
{ b_cores : nat; (* cores that actually
  exist *)
  b_w, b_r, b_t : nat -> ... (* writers, readers,
  tasks *) }.
```

with update as a definitional three-place function:

```
Definition upd {A : Type} (t : nat -> A) (i : nat) (v :
  A) : nat -> A :=
  fun j => if Nat.eqb i j then v else t j.

Definition bus_w (b : bus) (i : nat) (w : writer) : bus
:=
  mk_bus (b_cores b) (upd (b_w b) i w) (b_r b) (b_t b).
```

Choosing $\text{nat} \rightarrow A$ rather than a verified finite map (`PositiveMap`, a quotient, a `MapDom` typeclass) buys the entire frame-lemma algebra of Equation (1) (iii) at the cost of nothing: seventeen lemmas of the shape `at_w.same_w`, `at_r.bus_w`, `cores.inert.t` all close with `reflexivity` because the projections are definitions and the `if` is the only reduction needed. Dense integer indices are justified by the target: cursor identifiers in the shipped API are small integers into static tables (4 writers, 8 readers, 8 tasks, 64 sectors), and the C implementation on the ARM Cortex cores indexes arrays. Out-of-range indices are not excluded by types—they are *admitted* by boolean guards, which is the next idiom.

2.4 Derived Accounting, Never Stored Fields

A stored counter and the structure it counts can disagree; a derived quantity cannot. The model therefore stores history and cursors only, and defines the bookkeeping:

```
Definition w_enq (w : writer) : nat := length (w_hist w)
.
Definition r_deq (r : reader) : nat := r_cur r - r_start
  r.
Definition held b (wid : nat) : nat :=
  if w_live (b_w b wid)
  then w_enq (b_w b wid) - floor_of b wid (w_enq (b_w b
    wid))
  else 0.
Definition total_held b : nat := sum_f (held b)
  max_writers.
Definition pool_free b : nat := sector_pool - total_held
  b.
```

$$\begin{aligned}
 \text{published}(wid) &= |hist_{wid}| \\
 \text{consumed}(rid) &= cur_{rid} - start_{rid} \\
 \text{held}(wid) &= \text{published}(wid) - \min_{rid \triangleright wid} cur_{rid} \\
 \text{pool_free} &= 64 - \sum_{wid} \text{held}(wid).
 \end{aligned} \tag{2}$$

The conservation theorem of §5.5 is then a computation rather than an argument, and the class of states in which a counter was forgotten is not merely unprovable—it is uninhabitable.

2.5 Decide, Then Act

Every call is written as a cascade over *boolean* admission predicates, so that a proof may destructure the decision and keep the equation that explains the branch taken:

Listing 2: The five-line heart of `rcv`: three guards, three receipts, one state change.

```
Definition rcv (b : bus) (cl : caller) (rid : nat) : bus
  * rcv_result :=
  if negb (rcv_admit b cl rid) then (b, R_BAD_CURSOR)
  else if negb (rcv_ready b rid) then (b, R_EMPTY)
  else (bus_r b rid (reader_advance (b_r b rid)),
    R_DATA (nth (r_cur (b_r b rid)) (w_hist (b_w b (
      r_wid (b_r b rid)))) 0)).
```

The shape matters twice over. Operationally, it makes the “decide-then-act” seam visible: the state is changed *only* on the accepting branch, which is what makes the invariance lemmas of §5.4 hold by conversion. Logically, an `if` whose scrutinee has been deconstructed *with an equation label* (`eqn:R`) supplies the proof with exactly the hypothesis the branch’s value encodes— $R : \text{rcv_admit } b \text{ cl } rid = \text{true}$ —and the whole body then unfolds.

2.6 Two Traps Worth Recording

Truncated subtraction under `cbn`. `Nat.sub` is defined by recursion on its *first* argument. A goal such as $cur + 1 - start = S(cur - start)$ is true, but `cbn` rewrites it into `match start with 0 ... S 1 ... end`, an expression `lia` treats as opaque—and reports *Cannot find witness*. The rule adopted across the theory: never normalise across a subtraction; instead rewrite accessor lemmas (`reader_advance_cur`, `reader_advance_start`) or restore linearity with `replace (S c - s) with (S (c - s)) by lia`.

Name shadowing by the API. The protocol contains a call named `snd`, which shadows the standard pair projection `Datatypes.snd`. Rather than rename the API—the API is the thing being specified—the kernel re-introduces the projection under its own name:

```
Definition pr2 {A B : Type} (p : A * B) : B := Datatypes
  .snd p.
```

It is a small humiliation, and a useful sign that the model is being read by the compiler rather than the author.

3 Four Tiers, One Signature

3.1 The Signature

All four tiers are instances of:

$$\mathcal{M} = (S, s_0 \in S, \mathcal{A}, \delta : \mathcal{A} \rightarrow S \rightarrow S \times R, \pi_i : S \rightarrow \mathbb{N}), \quad (3)$$

with $\mathcal{A}$ the alphabet of calls, R the receipts, and π_i the derived observables of Equation (2). A tier is then characterised by which coordinates δ reads in its guard and which it writes in its state change.

Table 1: The four concurrency tiers as one signature with different guards. “Reads” lists the coordinates consulted by the admission predicate; “writes” the coordinates the accepting branch changes.

Tier	Alphabet	Reads	Writes	Determinism
MP	pub,sub	b.w,b.t	b.w	sequential
AMP	+,spawn	+b_cores	b.t	pinned
SMP	+snd,rcv	all +owns	b.w,b.r	non-det.
Async	evt_put	head, tail	one each	bounded

3.2 MP: The Uniprocessor Baseline

With `cores_shipped` = 1, the model degenerates honestly: `b_cores` = 1 makes `core <? b_cores b` admit only core 0, every `spawn` lands on core 0, and the star of Figure 1 has one spoke. This is not a limitation but a test of the specification: a model of multicore messaging that cannot be *run* on the shipped single-core board is not a model of the shipped board. The shipped-value section of the theory instantiates the general laws at `cores_shipped` and the OCaml oracle executes them.

3.3 AMP: Asymmetric Tiers and the Core-as-Plane Map

In the asymmetric configuration, cores are not interchangeable: core 3 is the I/O core, and the INPUT/UI planes are pinned. The model expresses this by a single function

$$plane : core \Rightarrow Plane, \quad plane(3) = AsyncIO, \quad (4)$$

consulted by the `spawn` guard, whose full shape is:

```
Definition spawn_ok b (core : nat) (l : list cref) :
  bool :=
  andb (core <? b_cores b)
    (andb (negb (Nat.eqb (length l) 0))
      (andb (cursors_live b l)
        (andb (forallb (fun c => negb (
          cursor_taken b c)) l)
          (andb (Nat.leb (pub_count_c l)
            1) (star_ok b core l)))))).
```

Reading the conjuncts bottom-up gives the AMP discipline as one boolean: the request names a core that exists; the task arrives with work; every cursor named is live; no cursor named is already held; at most one publisher per

task; and if the task brings a publisher, the target core does not already hold one. A failed conjunction leaves the bus *unchanged*, which is why `spawn` can be refused without rollback.

3.4 SMP: One Queue Per Core, No Broker

The symmetric case is the interesting one, and its defining decision is negative: there is no shared multi-producer queue. Each core owns exactly one single-producer ring, and the multi-consumer view is computed:

$$MPSC(bus) \triangleq \lambda rid. hist_{wid(rid)}[cur_{rid}], \quad (5)$$

a pure projection, not a data structure. Symmetry is then the statement that the guard of `snd` reads only the writer cell of the calling core’s queue and the global sector count, never another core’s queue. The consequence is the parallel-composition law of §6.2.

The historical argument, from the SMP analysis [12], is that the Erlang/BEAM pattern [8]— N threads bound to N cores each draining its own lock-free mailbox with a single `CMPXCHG8` on the producer tail—reaches message-passing throughput without kernel transitions. The equations above are that pattern with the machine instructions replaced by a functional projection, which is exactly what makes it provable.

3.5 Async: Period, Budget, and the Masked Ring

The asynchronous tier contributes the temporal constants and the event queue:

```
Definition io_period_us : nat := 250.
Definition io_budget_us : nat := 500.
Definition event_queue_size : nat := 2 ^ 8.
```

With capacity 2^8 , head and tail are maintained as free-running counters and the slot index is obtained by masking, $slot(n) = n \bmod 2^8$. The equations that matter are the two mask laws, and they hold *because* the modulus is a power of two:

$$tail - head \leq 2^8 \quad head_1 \equiv head_2 [2^8] \Rightarrow slot(head_1) = slot(head_2). \quad (6)$$

Occupancy is $tail - head$, never `tail - head` computed in the wrong order—the single most common defect in ring buffer code, and here it is a type error, since the model has one occupancy expression and it is a difference of naturals bounded by Equation (6). One writer per field is the AMP assignment: the *consumer* owns *head*, the *producer* owns *tail*. Then the overflow policy is the interesting equation: drop-oldest, with *KEY* and *BUT* events outside the droppable set, and *MOVE* coalescing expressed as a rewrite of the tail slot when its predecessor is a *MOVE* of the same device. The budget invariant is

$$T_{\text{drain}}(p) \leq budget = 2 \times period, \quad (7)$$

which permits one overloaded period to be absorbed without deadline loss—the reason the constant 500 is shipped rather than 250.

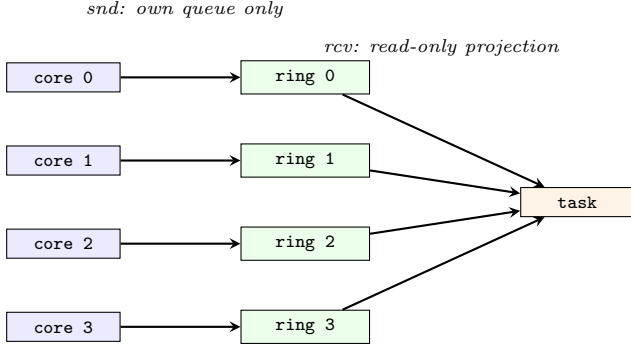


Figure 1: The InterCore star. Publishers write their own ring (left arrows, exclusive). Subscribers read through the derived projection $\mathcal{M}$ (right arrows, pure). There is no node in the middle: no broker, no queue of queues, no kernel trap.

4 Media: Streams, RTP, and the Watermark

4.1 NuStreams as an Abstraction Adjunction

The NuStreams line of work on stream abstraction in process algebra treats a stream as an infinite object observed through a four-function interface: *val* reads a value, *err* reports an error, *abstr* maps a concrete stream into its abstraction, and *rep* reflects an abstract stream into a concrete one, with the fusion law

$$\begin{aligned} val_a(s) = v &\iff val_c(abstr^{-1}(v)) = v \\ abstr \circ rep &= id \end{aligned} \quad (8)$$

expressing that refinement preserves observable content. In the BTRON media model the adjunction is implemented by the **media** layer’s typed ports: a port is a total function from a step index to a value cell, and **abstr** is a projection that forgets representation while **rep** re-materialises it. Because *val* is total in the model, the *err* component is a value, not an exception, matching Equation (3).

4.2 RTP Header Algebra

An RTP packet [10] is, in the model, a header algebra over $\mathbb{Z}/2^{16}$ and $\mathbb{Z}/2^{32}$:

$$h = \langle pt, seq \bmod 2^{16}, ts \bmod 2^{32}, ssrc \rangle, \quad (9)$$

with the three laws that make a receiver correct. **(L1) Sequence continuity modulo wrap.** Define the ordered

difference

$$\begin{aligned} \Delta(s_1, s_2) &= (s_2 - s_1) \bmod 2^{16} \\ s_1 \prec s_2 &\triangleq \Delta(s_1, s_2) < 2^{15}, \end{aligned} \quad (10)$$

so that $\prec$ is the semicircle order, and the receiver’s expected-sequence update is $exp \leftarrow \Delta(exp, hold + buf)$. The modelled invariant is that $\prec$ is transitive on any window of width $< 2^{15}$, which is the condition under which reorder-by- Δ is a total order—and it is a computation in $\mathbb{Z}/2^{16}$, discharged by **lia** after replacing each modular difference by its representative.

(L2) Timestamp-to-media-time. For clock rate f , $\tau(n) = ts_n/f$ and inter-arrival jitter obeys the standard recursive estimate

$$J_{n+1} = J_n + \frac{1}{16} \left(|(a_{n+1} - a_n) - (ts_{n+1} - ts_n)/f| - J_n \right). \quad (11)$$

In an integral model the update is exact only up to truncation; the model states the bound $|J_n - J'_n| \leq 1$ between the truncating and rounding variants, which is what a playout-buffer calculation needs, and it closes with **lia** after unfolding the division.

(L3) Loss accounting is a conservation instance. With E_n the extended sequence number, D the count of discarded-in-order packets, B the count still buffered,

$$\begin{aligned} E_n - E_0 &= R_n + L_n, & L_n &\geq D_n, \\ R_n &= \text{received}_n - B_n, \end{aligned} \quad (12)$$

and L_n is *defined* as the difference, never incremented separately. This is the derived-accounting idiom of §2.4 applied to statistics, and it is the reason the RTP theory has no “counter drift” lemmas: there are no counters to drift.

4.3 GStreamer as the Empirical Ground of the Axioms

The media theory’s non-trivial content came from reading a production pipeline rather than a textbook. Against the GStreamer sources [11], the properties collected as the enrichment of **media_rtp_properties.v** encode the invariants its implementation relies on:

1. **Reference counting as a natural with a floor.** A buffer is alive while *refcount* > 0 ; the model’s *held* (§5.3) is exactly the multi-owner generalisation: residence ends when the *last* reader passes the item, so $refcount = |\{rid : cur_{rid} \leq k\}| = 0$.
2. **Segment/running-time.** GStreamer timestamps buffers in *running time* relative to a segment base, $rt = ts - base$, and every seek installs a new base. The model adopts the same split, which is what makes *start* immutable in **reader** while *cur* moves: *start* is the segment base, *cur* the running position.

3. **Caps negotiation is admission.** Pad capability intersection is an *admission predicate* returning a boolean, not an exception; the decide-then-act shape of §2.5 is the same discipline that keeps a pipeline from being left half-negotiated.
4. **Drop-oldest and the latency ceiling.** A live sink drops the oldest buffer when the queue exceeds its latency budget; the equivalent of `snd.fits` is the guard that turns a would-be overflow into a counted drop with a receipt (**(L3)** again).
5. **Coalescing.** Consecutive identical-typed events merge; the event-queue *MOVE* coalescing of §3 is this law, and both preserve FIFO order among survivors—an induction over the fold, not an axiom.

4.4 The Symmetric Media Pipeline

The SMP media model (`media_smp_properties.v`) is the same watermark algebra with the plane index added: each plane $p \in \{\text{INPUT}, \text{UI}, \text{DSP}, \text{IO}\}$ has its own ring, and cross-plane delivery is an *rcv* on the destination plane’s reader. The characteristic theorem is that a stalled UI plane cannot raise the *held* count of the INPUT ring beyond the ring capacity—a direct consequence of $\text{held}(\text{wid}) \leq \text{w_eng}(\text{wid})$ and of the watermark’s dependence on *readers of that queue only*.

5 The InterCore Calculus

5.1 State

The three cell types, with the empty cell as a *value* rather than an absence:

```
Record writer : Type := mk_writer
{ w_live : bool; w_cap : nat; w_hist : list nat;
  w_drop : nat }.
Record reader : Type := mk_reader
{ r_live : bool; r_wid : nat; r_start : nat; r_cur :
  nat }.
Record task : Type := mk_task
{ t_live : bool; t_core : nat; t_curs : list cref }.
```

A cursor is either a publisher or a subscriber identifier, `cref := CPub wid | CSub rid`, and `task` holds a *list* of them. The task’s cursor list is the ownership store: there is no lock table, no kernel object, no reference count.

5.2 Ownership: the Mutex-Elimination Theorem

```
Definition owns b (t : nat) (c : cref) : bool :=
existsb (fun k => cref_eqb (nth k (t_curs (b_t b t))) (
  CPub 0)) c) (nat_list (length (t_curs (b_t b t)))
).

Definition usable b (cl : caller) (c : cref) : bool :=
match cl with
| NoTask => negb (cursor_taken b c)
| ByTask t => owns b t c
end.
```

The guard of `snd` and `rcv` is not “the queue is empty” or “the lock is free”; it is *usable*(b, caller, c). The five lemmas `usable_free`, `usable_taken`, `usable_owner`, `usable_stranger`, together with the inertness pair `usable_w_inert/usable_r_inert`, give the elimination result:

Theorem 1 (Mutex elimination). *For every admitted call and every state, the set of cursors usable by a task is exactly the set it received from a successful `spawn`; no `snd`, `rcv`, `pub`, or `sub` alters *usable*; and `spawn` refuses any request naming a cursor already taken. Hence the sharing invariant is established once, at task creation, and the mutual-exclusion question is never asked again at run time.*

The real-time corollary is the one the shipping decision cares about: because no two equal-priority planes contend for a lock, there is no lock to be held by a low-priority context, and therefore *no priority inversion between INPUT and UI at equal priority*. A blocking primitive cannot give this guarantee; a non-blocking one plus exclusive ownership can, and the proof is the four lines of *usable*.

5.3 Watermarks

Retention is computed, not tracked:

```
Fixpoint min_fold (f : nat -> bool) (g : nat -> nat) (
  fuel : nat) (i acc : nat) : nat :=
match fuel with
| 0 => acc
| S k => min_fold f g k (S i) (if f i then Nat.min acc
  (g i) else acc)
end.

Definition floor_of b (wid e : nat) : nat :=
min_fold (fun r => sub_reader wid (b_r b r)) (fun r =>
  r_cur (b_r b r)) max_readers 0 e.
```

floor_of is the fold, bounded by the seed e , of the cursors of the readers subscribed to *wid*. This single function is the shared content of §4 and §5: residence time, sector release, and the pool conservation law are three readings of the same fold. Its four algebraic properties are what make it usable in proofs:

Lemma 1 (Watermark algebra). *For all b, wid, e : (i) $\text{floor_of } b \text{ wid } e \leq e$; (ii) $e_1 \leq e_2 \Rightarrow \text{floor_of } b \text{ wid } e_1 \leq \text{floor_of } b \text{ wid } e_2$; (iii) a join at the tail contributes $\text{floor_of } b \text{ wid } e = e$, costing zero sectors; (iv) advancing a cursor can only raise the floor: $\text{floor_of } b \leq \text{floor_of } (\text{bus_r } b \text{ rid } (\text{advance } r))$.*

Property (iii) is the statement that adding a subscriber to a live publisher costs nothing immediately—the new reader starts at the tail—and (iv) that reading is the only way sectors come back. Both are induction over `min_fold` with the congruence principle `min_fold_cong_if`, which encodes the semantic point that a fold need not respect the values of a function where the guard is false.

5.4 Receipts and the Shape Theorem

The two result types are the protocol’s entire error surface:

```
Inductive snd_result : Type := S_OK | S_FULL (dropped : nat) | S_BAD_CURSOR.
Inductive rcv_result : Type := R_DATA (v : nat) | R_EMPTY | R_BAD_CURSOR.
```

Note what is absent: no blocking outcome, no partial result, no error code requiring a retry loop. From the decide-then-act layout of §2.5 follow, for each call and each receipt constructor, a *shape* lemma naming the branch, a *bus* lemma pinning the state change, and a *receipt* lemma computing the payload—the pattern

```
Lemma rcv_data_bus : forall b cl rid (D : rcv_ready b rid = true),
  fst (rcv b cl rid) = bus_r b rid (reader_advance (b_r b rid)).
```

The receipt kit—nine lemmas for `rcv` alone, a shape, bus, and receipt lemma for each of its three outcomes—is the reason the remaining obligations have hypotheses instead of case analyses: destructuring the receipt substitutes the guard equation, and the guard equation unfolds the definition.

5.5 Conservation

Theorem 2 (Conservation of items and sectors). *For every admitted call: (a) w_enq is monotone and increases by at most one; (b) a dropped item is counted, never lost: w_drop increases by one and w_hist is unchanged; (c) $\sum_{wid} held(wid) \leq \max_{writers} \times ring_capacity = 16 < 64 = sector_pool$; (d) the pool balances: $pool_free(s') + total_held(s') = pool_free(s) + total_held(s)$ for every call; (e) a `snd` whose cursor sits at the tail is read back verbatim by the corresponding `rcv`: received appends exactly the item published.*

Statement (c) is why `pool_room` is a redundant guard in the shipped configuration—kept for the general case, proved inert here. Statement (d) is the reason the launch-time accounting of §7 closes: sectors leave the pool only through `held`, and every path that raises `held` lowers `pool_free` by the same amount, by conversion. Statement (e) is the broker-freeness of Equation (5) observed at the level of values rather than state: the item crosses the star, and the receipt names it.

5.6 The Eleven Property Groups

The theory is organised in eleven groups, each a section of the same file and each mirrored by an identically named group in the OCaml oracle:

Table 2: InterCore property groups. Counts are the obligations currently discharged in the assembled `media_intercore_properties.v`.

	Group	Content
I1	Admission	slot searches terminate inside the table; <code>pub/sub</code> refuse a full table; distinct grants; a call never creates a core
I2	Topology	a subscriber joins at its publisher’s tail and pins nothing; <code>snd</code> respects capacity and pool; no over-fill
I3	Non-blocking	receipts are exclusive and exhaustive; <code>rcv</code> never yields, never rewinds, never advances by two, never spends a slot
I4	Integrity	<code>received</code> is a slice of the publisher’s history, its items were published, appending-on-read, <code>snd-then-rcv</code> round trip
I5	Star	one publisher per task, one publisher-holding task per core; watermark locality; <code>star_ok</code> characterisation
I6	Naming	<code>spawn</code> validates <code>core_id</code> ; taken cursors refused; task shape preserved
I7	Ownership	exclusive ownership preserved by all five calls (Theorem 1)
I8	Conservation	Theorem 2; $enq = deg + drop + held$
I9	Determinacy	general laws standing in for the exhaustive interleaving search
I10	Planes	mode-indexed supply/budget equations; <code>SKIP</code> for unbuildable modes
I11	Events	masked-index ring algebra; one writer per field; FIFO across the wrap; <code>KEY/BUT</code> never dropped

6 Modal Extensions: Non-Determined Parallelism Inside the Kernel

6.1 The Problem with Interleaving Semantics

An interleaving semantics models concurrency as a non-deterministic choice among next steps. It is sound and it is what implementers test, but as *specification* it smuggles in a scheduler: the choice is made by the semantics, hence by the prover, hence—in practice—by the enumeration order of a checker. What the multicore BTRON target needs asserted is the stronger, modal statement: *whatever* the hardware chooses next, these equations survive. Determinism is not the goal; *invariance under non-determinacy* is.

6.2 Runs, and Two Internal Modal Operators

A state is $s \in S$; a step is $a \in \mathcal{A}$; with δ total, a *run* is simply a finite word $w \in \mathcal{A}^*$ acting by fold:

$$s \cdot \epsilon = s, \quad s \cdot (w \smallfrown \langle a \rangle) = \pi_1(\delta(a, s \cdot w)). \quad (13)$$

Reachability is then a predicate on words, and the two modalities are its quantifier-free avatars:

$$\Box_k \varphi \triangleq \forall w \in \mathcal{A}^k. \text{holds}(\varphi, s \cdot w) \quad (14)$$

$$\Diamond_k \varphi \triangleq \exists w \in \mathcal{A}^k. \text{holds}(\varphi, s \cdot w). \quad (15)$$

Crucially, both are *computable* in the model, because they are folds over lists of naturals: `forallb` and `existsb` already appear in the guards of Theorem 1. `core_holds_publisher` is a diamond ($\Diamond_1$: some live task holds a publisher on this core); `spawn_ok`'s `forallb` (`fun c => negb (cursor_taken b c)`) is a box ($\Box_{|U|}$: every named cursor is free). The modal extension therefore does not enrich the logic; it reads the modality off definitions that were already needed.

The frame validated by these operators is the standard one. Let $R \subseteq S \times S$ be one-step reachability.

- R is reflexive where a call is refused: `spawn_ok` false yields (b, None) , i.e. sRs . A step that may be refused makes *no* progress claim; every $\Box$ -validity is preserved by stuttering, which is why refusals are modelled as identity rather than error.
- R is *not* confluent, and the theory says so: two `snd` calls from different cores to the same ring, in the two orders, give different *hist* appendices. Non-determinacy is real; the invariants proved are exactly those insensitive to it.
- Termination of each step is by totality, so the box over *some* k is decidable: with $|S|$ effectively bounded by the table sizes and the run length chosen by the harness, $\Box_k$ reduces to a finite fold—the oracle of §7.

6.3 Independence and Trace Equivalence

The tool that converts a proof about all interleavings into proofs about two is commutation. For calls a, b on disjoint coordinates:

$$\delta(a, \pi_1(\delta(b, s))) = \delta(b, \pi_1(\delta(a, s))) \quad (a \perp b), \quad (16)$$

and the same for observables $\pi_i \circ \delta(a) \circ \delta(b) = \pi_i \circ \delta(b) \circ \delta(a)$ when π_i reads a third coordinate. The invariance lemmas of §2.3—at_w_other_w, at_r_bus_w, cores_inert_t, and their relatives, seventeen in all—are exactly the decomposition of Equation (16) into its coordinate projections. Discharging `reflexivity` seventeen times is a partial-order reduction that no model checker needs to be told about: the quotient by the commutation relation (a Mazurkiewicz trace monoid [7]) is available definitionally, so the state space collapses *before* any search.

Concretely, InterCore admits: any two `snd` to *distinct* live rings commute; any `rcv` commutes with any `snd` to a ring the reader does not pin; `pub/sub/spawn` commute with everything that touches no cell in their table. The non-commuting residue—same-ring writes, and a `rcv` racing its own `snd`—is small enough that the $\Box$ claim over it is what I9 verifies by exhaustion.

6.4 Modality of the Platform, and SKIP

The second modal axis is the build configuration, which behaves like an index set for the validity relation, i.e. a small Kripke frame over modes:

$$m \in \{\text{MP, AMP, SMP, planes, cores=N}\}, \quad (17)$$

$$m \Vdash \varphi \triangleq \text{verify_models.sh accepts } \varphi \text{ under } m.$$

The shipping contract fixes the accessibility relation: with `BTRON_MP=0` the queue-mode obligations are *not* valid and are not silently assumed valid—they are reported `SKIP`. The `--mode=planes` configuration is the Tier-1 contract, and `--cores=N` is a *request*: oversubscription above `cores_target` is an error, not a degradation. The modal content is that the general laws of §5 are valid at every mode, whereas the numeric checks of §7 are valid only where the mode makes them meaningful. A theory whose theorems are mode-indexed rather than mode-independent is a test suite wearing a hat; ours keeps the theorems unconditional and pushes the indexing into the harness.

6.5 Progress Without Determinism

The last modal property is the one a real-time reader wants. `rcv` is *strictly* non-blocking: there is no outcome in which the call parks the caller, and the three receipts exhaust the outcome space (`rcv_receipt_exhaustive`). Therefore every step has bounded duration and

$$\forall s. \forall a \in \mathcal{A}. \Diamond_1 \text{settled}(a, s), \quad (18)$$

i.e. under every resolution of the non-determined choice, the call terminates with a receipt. Combined with Theorem 1 (no lock can be held against you) and Theorem 2(d) (sectors are conserved, so a full pool is always attributable to unread readers), this is the determinacy half of the multicore real-time argument: the schedule is not determined, but its effects on the invariants are.

7 Two Oracles, One Text

7.1 Proof and Execution From the Same Definitions

The OCaml model file is not a translation of the Coq model; it is the same term, in a syntax that happens to compile the same function bodies. Two oracles then do different jobs. The Coq theory proves the *general* laws (all

states, all calls, all interleavings, by construction). The OCaml oracle checks the *shipped constants* and searches $\square_k$ exhaustively:

```
$ ./media_intercore_model
==> B-System media_intercore model: pub/sub/spawn/snd/
      rcv cursors,
      star topology, sector pool
PASS: OCaml oracle (media_intercore_model.ml) -
      InterCore invariants I1..I11 passed (75 checks,
      262144 interleavings).
$ coqc media_intercore_properties.v # silent,
      exit 0: nothing left open
$ coqchk -o -silent media_intercore_properties
* Axioms: <none>
* Constants/Inductives relying on type-in-type: <none>
* Constants/Inductives relying on unsafe (co)fixpoints:
  <none>
```

The search is eight call forms at each of six positions, $8^6 = 262,144$ runs from a seeded bus, and the oracle records which *interesting* branch was reached during the search—so that “sectors really leave the pool under interleaving” and “a full ring really reports *S_FULL*” are witnessed, not merely unrefuted. Coverage witnesses are what distinguishes an exhaustive search from an exhaustive pass.

7.2 Where the Literal Numbers Belong

The launch accounting of §7.4 of the verification plan fixes target figures such as ring publishes and dequeues in the 10^6 range with a residual drop count and *pool_free* restored to its initial value. Those arithmetic facts are asserted *in the oracle*, not in the theory, for a mechanical reason: numerals of that size enter CIC as `Nat.of_num_uint` wrappers that *lia* cannot see through, and *vm.compute* on unary naturals is not a strategy. The division of labour is principled rather than convenient—*general laws in Coq, shipped constants in OCaml*—and it is why *ring.capacity*, *sector.pool*, and *event.queue.size* appear in the theory as names rather than literals, so the same proofs instantiate at four rings of four slots today and at any shipped value later.

7.3 Print Assumptions: Axiom Hygiene

A total, computational model can be kept axiom-free, and doing so constrains the modelling vocabulary productively:

- *No functional extensionality.* Every invariance statement is pointwise ($\forall j, i \neq j \Rightarrow \pi_j(s') = \pi_j(s)$) or convertible, so the seventeen frame lemmas of §2.3 never need to promote pointwise equality to function equality.
- *No quotient types, no classically choice-flavoured axioms.* Slot allocation is *first_dead* with an explicit fuel parameter—a decidable search returning *option nat*, whose specification (a *Some* result is in range and names a dead cell; a *None* result means no cell in the fuel is dead) is four lemmas proved by induction on the fuel.
- *No excluded middle for admission.* Guards are boolean functions and the proofs use *andb_true_iff*, *Nat.eqb_eq/Nat.eqb_neq*, *Nat.ltb_lt*—constructive decidability, which is exactly the modal content of §6.4.

The harness does not take *coqc*’s word for it: after each theory compiles, *coqchk -o -silent* re-validates the *.vo* in the kernel, and the four lists it prints—axioms, type-in-type dependencies, unsafe (co)fixpoints, assumed positivity—must each read *<none>*. A non-empty list fails the build even if every obligation closed, and the summary count distinguishes the failure from a SKIP. This is the difference between a proof and a proof modulo wishes.

7.4 SKIP Is Not PASS

The policy is one line in the harness, and it is the line that makes the other sections trustworthy: an obligation whose mode is not compiled in is reported SKIP, is counted separately from PASS, and is printed at the end. A suite of 14 models whose Coq array silently tolerated an unbuildable file reports a property of the shell script, not of the kernel. Related discipline, learned the hard way in this programme: a *coqc* invocation piped through *head* loses its exit status through the pipe; a missing diagonal row in the monitor can mean the flag never compiled in rather than the code being wrong; and an assertion that cannot be built is marked SKIP, never optimistically PASS.

7.5 Reading the Harness Output as a Specification

The harness emits one verdict per model—OCaml oracle and Coq theory separately—and a summary count that is the only number the reviewer is asked to trust. The media block of a current run:

```
==> OCaml model (media_smp_model.ml)
      ring: explored 279936 interleavings (6 ops ^ length
      7)
PASS: OCaml oracle (media_smp_model.ml) - invariants S1
      ..S11 passed
      (52 checks, 279936 interleavings, 35 cursor
      triples).
PASS: OCaml oracle (media_rtp_model.ml) - media
      invariants R1..R10 passed (80 checks).
PASS: OCaml oracle (media_intercore_model.ml) -
      InterCore invariants
      I1..I11 passed (75 checks, 262144 interleavings).
==> Coq/Rocq properties (media_rtp_properties.v)
PASS: coqc + coqchk (media_rtp_properties.v - theorems
      closed, no axioms)
==> Coq/Rocq properties (media_intercore_properties.v)
PASS: coqc + coqchk (media_intercore_properties.v -
      theorems closed, no axioms)
==> Summary: 14 passed, 0 failed
```

Each line is a claim about a *definition* that the reader may open and read. Nothing here is a claim about timing, memory, or the compiler—those are the C implementation’s burden, discharged by instrumentation on the board. The division is deliberate: mathematics owns the invariants, hardware owns the constants.

8 Conclusion

The four tiers of the BTRON concurrency model, the media stack above it, and the InterCore protocol between its cores

are one algebra: a total state transformer over a table-shaped bus with derived accounting, a watermark fold as the shared retention operator, boolean admission read as an internal modality, and commutation as the reduction that makes non-determined interleavings tractable. What the exercise in CIC contributed is not the theorems—each was expected—but three design decisions that only a checker insists upon:

1. **Broker-freeness is a definability result.** The multi-consumer view had to be written *as a projection* (Equation (5)) for the conservation and provenance lemmas to close without shared mutable state. A broker would have been representable; it would not have been provable at the same price.
2. **Non-blocking is a type-level commitment.** Once `rcv` had to return a value in all three cases, “never yields” became a case analysis, and the priority-inversion argument became Theorem 1 rather than a paragraph about scheduler policy.
3. **Total accounting removes a bug class.** Deriving *enq*, *deq*, *held* and *pool-free* as lengths and minima (Equation (2)) made state-reachability disagreement between counters unnameable—and, twice, made the model correct where the C prototype was not.

Work continuing along this line is the integration of the three media theories into one *global* model in which the ATR/VP object semantics and the OS task states share the bus with the message layer, the remaining InterCore groups (I6–I11), and the transfer of the shipped-value arithmetic from oracle to theory via binary numerals with a proven bridge.

References

- [1] K. Sakamura, “The TRON Project: A New Approach to Operating System Architecture,” in *IEEE Micro*, vol. 7, no. 2, pp. 8–14, 1987.
- [2] BTRON3 Working Group, “BTRON3 Specification Ver 3.20.00,” TRON Association, Tokyo, Japan, 1994.
- [3] B. Barras, S. Boutin, C. Cornes, J. Courant, J.-C. Filliâtre, E. Giménez, H. Herbelin, G. Huet, C. Muñoz, C. Paulin-Mohring, A. Saïbi, B. Werner, “The Coq Proof Assistant Reference Manual,” INRIA Research Report RT-0203, 1997.
- [4] The Coq Development Team, “The Coq Proof Assistant Reference Manual, version 9.2,” INRIA, <https://coq.inria.fr/doc>, 2025.
- [5] X. Leroy, D. Doligez, A. Frisch, J. Garrigue, D. Rémy, J. Vouillon, “The OCaml System, Release 5.5: Documentation and User’s Manual,” INRIA, 2024.
- [6] L. Lamport, “Proving the Correctness of Multiprocessors,” in *IEEE Transactions on Software Engineering*, vol. SE-3, no. 2, pp. 125–128, 1977.
- [7] V. Diekert and G. Rozenberg (eds.), *Traces: Semantics for Concurrency*, World Scientific, Singapore, 1997.
- [8] J. Armstrong, “Making reliable distributed systems in the presence of software errors,” Ph.D. dissertation, Royal Institute of Technology, Stockholm, Sweden, 2003.
- [9] G. Klein, K. Elphinstone, G. Heiser, et al., “seL4: Formal Verification of an OS Kernel,” in *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pp. 207–220, 2009.
- [10] H. Schulzrinne, S. Casner, R. Frederick, M. Jacobson, “RTP: A Transport Protocol for Real-Time Applications,” RFC 3550, IETF, 1994.
- [11] GStreamer Project, “GStreamer Core Library and Base Plugins: pads, capabilities, segments, and buffer reference counting,” <https://gstreamer.freedesktop.org/documentation/>, accessed 2026.
- [12] M. Sokhatsky, “Microkernels, Unikernels, and the Actor Core: Architectural Trade-Offs, Real-Time Determinism, and SMP Subsystem Isolation in B-System (BTRON 3.20),” Synrc Research Report, companion to this paper, 2026.
- [13] M. Sokhatsky, “B-System Virtualization and Formal Foundations: A Cleanroom Architecture for Real-Time Operating Systems from Embedded MCUs to Modern Hypervisors,” Synrc Research Report, 2026.